

Kang Open Banking

Security & Compliance Whitepaper

Version 4.16.4 · April 2026

A complete reference for security architects, auditors, and integrators evaluating Kang Open Banking for production deployment. Every claim in this document is independently verifiable against the public `/healthz` and `/oidc-config` endpoints.

1. Executive Summary

Kang Open Banking is a production-ready Open Banking platform serving the CEMAC region. It implements FAPI 1.0 Advanced (the strongest published OpenID Foundation profile for financial-grade APIs) end-to-end: PAR, JAR, PKCE-S256, and mTLS-bound tokens are required for all financial flows. AISP and PISP consent lifecycles follow the canonical Open Banking UK semantics.

The API is at version 4.16.4. The published Node SDK is at `@kangopenbanking/sdk v1.2.0`. Reviewers occasionally encounter unrelated stale npm packages (v0.1.0 with only a `hello()` method); those are not the official client. Always verify against `/oidc-config` and `/healthz`.

2. Architecture Overview

Request lifecycle for a regulated financial operation:

```
Client (TPP)
-> mTLS (RFC 8705) - client certificate presented
-> Reverse proxy (TLS 1.2+ termination, cert header forwarding)
-> PAR endpoint (RFC 9126) - pushed authorization request
-> Authorize endpoint (OIDC + PKCE S256)
-> Token endpoint (cert-bound access_token + refresh_token)
-> Resource server (AISP / PISP / Gateway / Banking)
-> Double-entry ledger + audit log + webhook dispatcher
```

3. Authentication & Authorization

Supported OAuth 2.0 grant types: `authorization_code`, `refresh_token`, `client_credentials`. The implicit and resource-owner-password grants are not supported - they are explicitly excluded by FAPI 1.0 Advanced. PKCE with S256 is mandatory for all public and confidential clients.

Dynamic Client Registration (RFC 7591) is exposed at `/dcr-register` with software-statement validation. Registered clients receive a `client_id` and a server-generated `client_secret` stored only as a SHA-256 hash.

4. Cryptography

Layer	Algorithm / Standard
Transport	TLS 1.2+ (TLS 1.3 preferred)
JWS signing	RS256, PS256, ES256
Hashing	SHA-256 (tokens, webhooks, API keys, PINs)
Symmetric encryption at rest	AES-256
Random sources	Web Crypto getRandomValues() / Deno equivalent

5. Token Lifecycle & Rotation

Access tokens are short-lived and certificate-bound when mTLS context is present. Refresh tokens rotate on every use; reuse is detected and revokes the entire token family per OAuth 2.1 best practices. The JWKS endpoint serves both current and previous keys during rotation overlap windows. Quarterly scheduled rotation is supplemented by manual rotation via the admin console.

6. Webhook Integrity Model

Outbound webhooks are signed with HMAC-SHA256 over the canonical body and a timestamp. The signature is delivered in the x-webhook-signature header. Receivers should reject signatures whose timestamp is more than 5 minutes from current time. Delivery is retried up to 7 times with exponential backoff. Idempotency keys on receiving endpoints prevent duplicate side effects.

7. Audit Logging

Three tables capture the full event trail: `audit_logs` (all identity events), `security_audit_logs` (risk-scored events with IP/UA), and `consent_events` (AISP/PISP consent lifecycle). All inserts are append-only and protected by RLS; the `has_role()` security-definer function gates admin reads.

8. Regulatory Mapping

Standard / Regulation	Status	Scope
FAPI 1.0 Advanced	Implemented	PAR + JAR + PKCE-S256 + mTLS
OIDC Core 1.0	Implemented	Discovery, ID tokens, UserInfo
OAuth 2.1 (aligned)	Implemented	AuthZ code, refresh, client credentials
RFC 7591 (DCR)	Implemented	Dynamic Client Registration
RFC 9126 (PAR)	Required	Pushed Authorization Requests
RFC 9101 (JAR)	Required	JWT-Secured Authorization Request
RFC 7636 (PKCE)	Required	S256 only
RFC 8705 (mTLS)	Implemented	Cert-bound access tokens
PSD2 SCA	Aligned	Strong Customer Authentication
COBAC / BEAC	Compliant	CEMAC regulators
ISO 20022	Implemented	pacs / camt / pain
PCI DSS Level 1	Compliant	Card data via tokenization

9. Deployment Hardening Checklist

- Terminate TLS at a reverse proxy that forwards client cert headers (X-SSL-Client-Cert).
- Enable strict transport security (HSTS) with a minimum 1-year max-age.
- Rotate JWKS signing keys at least quarterly; rotate API client secrets at least annually.
- Integrate TOTP secret encryption with a dedicated KMS (AWS KMS / GCP KMS / Vault).
- Ship audit logs to an immutable WORM store (S3 Object Lock or equivalent).
- Configure rate limits per environment (60 req/min sandbox, 300 req/min production baseline).
- Run the /healthz probe from your monitoring system at least every 60 seconds.

10. Incident Response & SLA

Severity	Response	Resolution	Example
Critical	< 15 min	< 4 hours	Complete payment processing outage
Major	< 30 min	< 8 hours	Single payment channel degraded
Minor	< 2 hours	< 24 hours	Elevated latency on non-critical endpoint

11. Version History

- 4.16.4 - Added /healthz security posture endpoint, hardened /oidc-config caching (ETag + SWR), published this whitepaper, public /developer/security live verification page.
- 4.16.3 - Prior baseline.
- 4.15.0 - CEMAC universal bank integration (Wave 4 documentation).

Verify all claims live: <https://kangopenbanking.com/developer/security>

Direct probes: [/healthz](#) · [/oidc-config](#) · [/openapi.json](#)

Published per Standing Order P1 (Public First) and P10 (Living Docs).